EE5203 L08 Study Sheet

PWM, output compare, and input capture - Basri Erdogan

Essential question: One register is called *capture/compare*. When you write it, the pin makes a waveform; when the pin makes a waveform, the hardware writes it. What decides which direction it runs?

What you should be able to do

- Explain PWM: frequency from `ARR`, duty from `CCR`.
- Choose `PSC`/`ARR`/`CCR` for a required frequency and duty from 16 MHz.
- Hand a pin to a timer and prove the handover in `MODER`.
- Configure direct and indirect capture channels and read the timestamps.
- Recover period, frequency, and duty from captures — wrap-safe.

One channel, two directions

```
GENERATING (output compare)          MEASURING (input capture)
  you write CCR                        the pin's edge triggers a snapshot
  CNT counts up                        CNT counts up
  CNT < CCR   -> output high           edge arrives -> CNT copied into CCR
  CNT >= CCR  -> output low            CC1IF set; you read CCR
  CNT reaches ARR -> wrap              CNT free-runs to 0xFFFF and wraps
```

The direction bit is `CC1S` in `CCMR1`: `00` = output, non-zero = input. Same register file, same counter, opposite arrow.

Vocabulary table

Name	Job
`CCR1`…`CCR4`	compare value (you write) **or** captured timestamp (hardware writes)
`CCMR1/2`	`CC1S` direction, PWM mode, preload, input filter `ICF`
`CCER`	`CC1E` channel enable; `CC1P` output polarity / capture edge
`CC1IF` (`SR`)	a capture has landed
`CC1OF` (`SR`)	overcapture — a timestamp was overwritten before you read it
Direct mode	the channel listens to its own pin
Indirect mode	the channel listens to the *other* pin of the pair

PWM: the arithmetic

$$f_{\text{PWM}} = \frac{f_{\text{timer clock}}}{(\text{PSC}+1)(\text{ARR}+1)} \qquad \text{duty} = \frac{\text{CCR}}{\text{ARR}+1}$$

Requirement (16 MHz)	`PSC`	`ARR`	`CCR`	Result
1 kHz LED, 35 %	15	999	350	1 µs tick, duty in tenths of a %
50 Hz servo, 1.5 ms	15	19999	1500	1 µs tick — `CCR` **is** the pulse width in µs
200 Hz, 25 %	15	4999	1250	

The 100 % subtlety. The counter visits `ARR+1` states, so `CCR = ARR` gives `ARR/(ARR+1)` = 99.9 %, not full-on. True 100 % needs `CCR > ARR`.

One timer, one frequency. All four channels share `PSC` and `ARR`. A 1 kHz LED and a 50 Hz servo need two different timers.

The pin handover

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);   /* enables CC1E                */
TIM2->CCR1 = 350;                           /* HAL macro for the same write:
                     __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 350) */
```

CubeMX sets `MODER` bits for PA5 to `10` (alternate function) and selects AF1 for TIM2. Until that happens the pin still belongs to `HAL_GPIO_WritePin` and PWM does nothing visible — the classic silent failure.

Servo: the standard model

Pulse width	Angle (model)
1000 µs	0°
1500 µs	90°
2000 µs	180°

Real servos differ. Many turn less than 180° between 1000 and 2000 µs, and full travel often needs about 500–2400 µs. 1500 µs is the middle on almost every servo. Find the end points of your servo by testing, and clamp to them.

```
uint16_t servo_us(uint8_t deg) {        /* 0-180 deg -> 1000-2000 us */
    if (deg > 180) deg = 180;           /* clamp: never past an end point */
    return 1000 + (1000u * deg) / 180u;
}
```

Repeat every 20 ms. A pulse past an end point pushes the servo against its stop: it buzzes and draws a large current.

Capture: the arithmetic

```
/* at each rising edge: channel 1 direct, the only capture interrupt */
uint16_t now = TIM4->CCR1;
period_us = (uint16_t)(now - last);        /* cast: wrap-safe on 16 bits    */
high_us   = (uint16_t)(TIM4->CCR2 - last); /* CCR2 = falling edge, channel 2 */
last = now;                                 /* update last only at the end   */
```

$$f_{\text{signal}} = \frac{1}{\text{period}} \qquad \text{duty} = \frac{\text{high_us}}{\text{period_us}}$$

- **The cast is not decoration.** On a 16-bit timer the subtraction must be done in 16 bits: captures 64000 then 1500 give `(uint16_t)(1500 - 64000)` = **3036**, the true elapsed count. Without the cast you get a large negative number.
- **Direct vs indirect.** CH1 direct listens to its own pin (TI1); CH2 indirect listens to TI1 as well, so both channels watch one signal on opposite edges. CH2 configured *direct* would listen to PB7 instead — and capture nothing.
- **Range.** A 16-bit timer at a 1 µs tick measures honestly up to 65 536 µs 65.5 ms, i.e. signals down to about **15.3 Hz**. Slower than that, the counter wraps more than once between edges and the number lies silently.

SFR evidence

Claim	Evidence
the duty is what you set	`TIM2->CCR1` = 350; write 700 and the brightness steps
the pin belongs to the timer	`MODER` bits 11:10 = `10`
the channel output is live	`CCER` bit 0 (`CC1E`) = 1; clearing it kills the output
hardware is timestamping	`TIM4->CCR1` changes between pauses with no code running
the edge selection	`CC1P` in `CCER`; flip it and the captures move
a capture was lost	`CC1OF` in `TIM4->SR`

Common mistakes

- Assuming 84 MHz from an online example — our board runs 16 MHz.
- Pin never handed over: `MODER` still `01`.
- Confusing `CCR` and `ARR`: “changing the frequency” by writing `CCR`.

- Expecting `CCR = ARR` to be 100 %.
- One timer for two different frequencies.
- Subtraction not cast to `uint16_t`.
- CH2 left in direct mode, so duty comes out nonsense.
- `Start_IT` never called — configured, armed, but `CCR1` frozen.
- No input filter on a noisy line: every ring counts as an edge.

Mastery self-check

☐ I can derive `PSC`/`ARR`/`CCR` for a stated frequency and duty.
☐ I can explain why `CCR = ARR` is 99.9 %.
☐ I can state what `MODER` reads once a pin is handed to a timer.
☐ I can say who writes `CCR` in each direction, and when exactly.
☐ I can explain direct vs indirect and what each channel listens to.
☐ I can compute period and duty from two or three captures, wrap-safe.
☐ I can state the slowest signal a 16-bit timer measures at a 1 µs tick.
☐ I can name the flag that tells me a capture was lost.

Five review questions

1. From 16 MHz, give `PSC`/`ARR`/`CCR` for a 2 kHz PWM at 60 %. Then say what changes if the requirement becomes 2 kHz at 60 % **on a servo timer already running at 50 Hz** — and why.
2. A student sets `CCR = 1000` with `ARR = 999` expecting 100 %, and reports the LED is full-on. Are they right? Explain what the comparison does.
3. Your PWM code is correct and the LED does not dim, but `HAL_GPIO_WritePin` still controls it. Name the register and the exact value you would check.
4. Captures on a 16-bit, 1 µs timer arrive at `64000` then `1500`. Compute the period two ways — with and without the `uint16_t` cast — and explain the difference in terms of modular arithmetic.
5. You measure a 10 Hz signal with a 16-bit timer at a 1 µs tick and get a plausible-looking but wrong period. Explain the failure and give two different fixes.

See the L08 worksheet for extended practice. Worked solutions are released after the practice window.